- necessary & sufficient conditions
- two's compliment
- toString()
  - Every class inherits from Object class in Java
  - there is an in-built toString() in Object class that prints the address of object

- System.out.println(o);
  ↓
  System.out.println(o.toString());

```
public class Student extends Object {
    String name;
    int age;

    public String toString() {
        return "Student{name=" + this.name
            + ", age=" + this.age + "}";
    }
}

Student s = new Student("John", 35);
println(s);  // Student{name=John, age:35}
```

```
public Student() {

}
```

- Propositin logic
  - $A \Rightarrow B$ → "⇒" doesn't indicate causality or chronology
  - ≡ if A, then B

- If x & y are even, then x+y is even.

- $A \Rightarrow B$ is true
  - Is B necessary for A? Yes
  - Is B sufficient for A? No
  - Is A necessary for B? No
  - Is A sufficient for B? Yes
- $B \Rightarrow A$ is true
  - B is sufficient for A
- $A \Rightarrow B$ is true & $B \Rightarrow A$ is true
  - B is both necessary & sufficient for A

- if $\underbrace{x \text{ is even}}_{A}$, $\underbrace{x^2 \text{ is even}}_{B}$

  $(A \Rightarrow B)$ $x = 2k \Rightarrow x^2 = 4k^2 = 2(2k^2) \Rightarrow x^2$ is even

  $(B \Rightarrow A)$ To prove: x^2 is even $\Rightarrow$ x is even
  ≡ x is odd $\Rightarrow$ x^2 is odd $\Rightarrow x = 2k+1 \Rightarrow x^2 = 2k_1+1 \Rightarrow x^2$ is odd

$A \Rightarrow B$
- Converse: $B \Rightarrow A$
- inverse: $\sim A \Rightarrow \sim B$

(converse & inverse: equivalent)
- Contrapositive: $\sim B \Rightarrow \sim A$

$A \Rightarrow B \equiv \sim B \Rightarrow \sim A$

$B \Rightarrow A \equiv \sim A \Rightarrow \sim B$

To prove: A iff B

$A \Leftrightarrow B$

$A \Rightarrow B$ and $B \Rightarrow A$

- Proof by contradiction

- $\forall x, P(x)$ is true → We need to prove it for every 'x' (to prove)
  - to disprove → We just need to find one counter example

- $\exists x, P(x)$ is true → finding one example is enough
  - to disprove → need to prove that not true for every

- $\sim[\forall x, P(x)] \equiv \exists(x), \sim P(x)$

  $\sim[\exists x, P(x)] \equiv \forall x, \sim P(x)$

- $\sim(A \Rightarrow B)$
  $\equiv A$ and $\sim B$

A	B	$A \Rightarrow B$	$\sim A$ or B	$\sim(\sim A$ or $B)$ A and $\sim B$
T	T	T	T	F
T	F	F	F	T
F	T	T	T	F
F	F	T	T	F

List l → empty list

l.get(0) ; // error

l.addLast(10) ; [10]

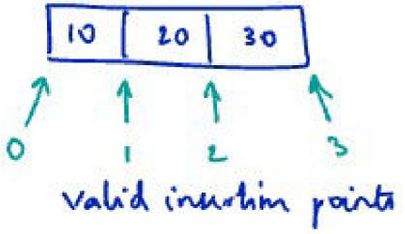
l.addLast(20) ; [10, 20]

l.addLast(30) ; [10, 20, 30]

l.size() ; 3

l.add(3, 50) ; [10, 20, 30, 50]

l.add(5, 50) ; // error



after adding, 50 should be at index 3

0 1 2 3

l.set(3, 40) ; [10, 20, 30, 40]

l.set(4, 100) ; // error

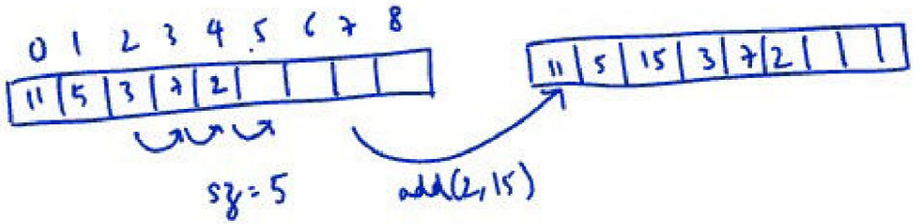
0 1 2

x = l.remove(2); // x = 30 l·[10, 20, 40]

l.remove(3) ; // error

l.clear()

· Things that can fail:
  · if array is full
  · if the given index is out of bounds
    valid index range is [0, size]
  · check for insertion at beginning & end.



add(2, 15)

1. increment size
2. Move elements from given index to index 'size-2', one step to the right (do it from right to left)
3. assign the given value to the given index

- `for ( int i = siz - 1 ; i >= index; i--)`
  `a[i+1] = a[i]`

  i indicates the index of elements that we need to move

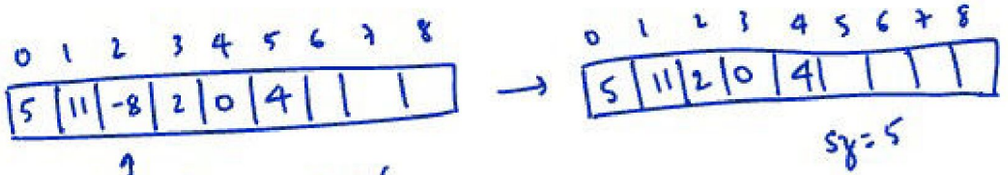
- Move everything from index to siz-1 one step to right. (from right to left)

```
i = siz-1
while (i >= index) {
    a[i+1] = a[i];
    i--;
}
```

```
while (true) {
    if (cond)
        break;
```

! cond

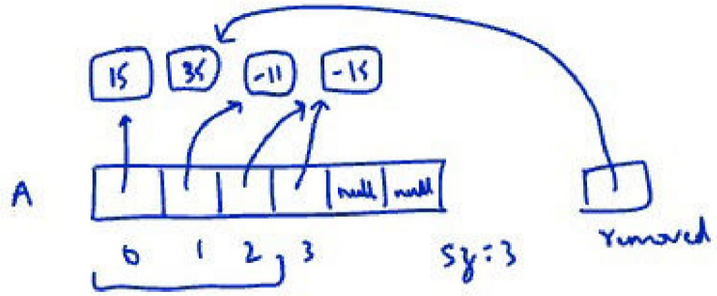
- remove (index)



- Elements from index+1 to siz-1 should be moved one step to the left (move from L to R)
- Decrement size
- Return the old element

Things that could fail:

1. Invalid index given
   Valid indices: [0, size-1]
2. Edge cases:
   - index = 0
   - index = size-1
3. Can't remove if array is empty. Covered by 1. as valid indices: [0, -1] => no valid indices



remove element at index 1

T removed: A[1]

A[1]: A[2]

A[2]: A[3]

sz--;

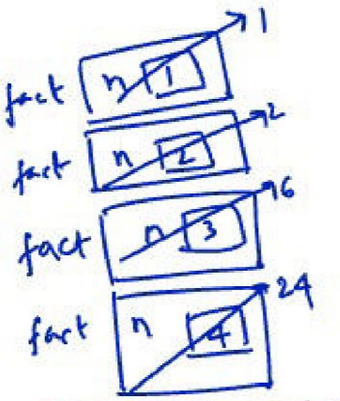
- Space complexity
  - We will calculate the amount of extra memory used, other than the input & output.
  - This includes the recursive stack space

```
int fact (int n) {
    if (n ≤ 1) return 1;
    return n * fact(n-1);
}

x = fact(4);
```

↓

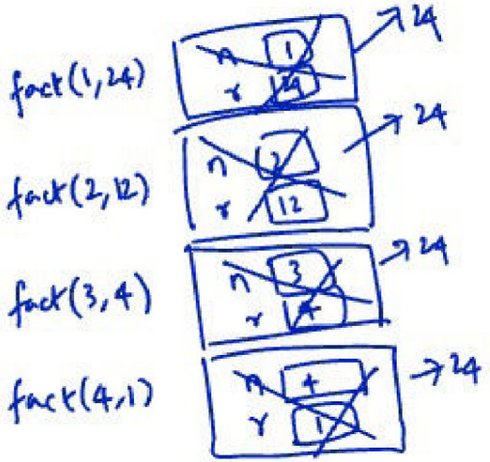
24



Tail recursion

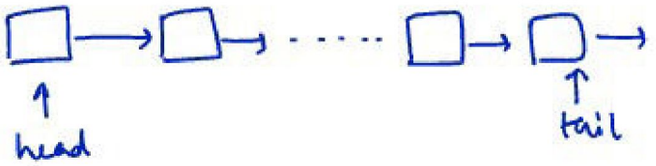
```
int fact(int n, int result) {
    if (n ≤ 1) return result;
    return fact(n-1, n * result);
}
```

```
int fact (int n) {
    return fact(n, 1);
}
```

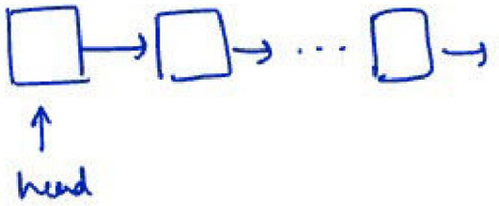


Singly linked lists:

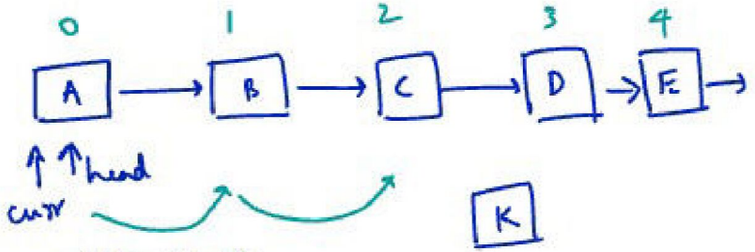
- advantage of using a tail pointer



- insertLast() would take $\Theta(1)$ instead of $\Theta(n)$
- getLast() would take $\Theta(1)$ instead of $\Theta(n)$
- removeLast() would still take $\Theta(n)$ because we can't get access to the last but one element



head = new Node(value, head)



add(3, "k")

- move curr "index-1" th times & insert the new node after curr

```
Node curr = head;
while(curr != null) {
    ...
    curr = curr.next;
}
```

```
for(i=0; i<k; i++) {
    ...
}
```

→ this loop runs 'k' times

add(int index, T value) in SLL

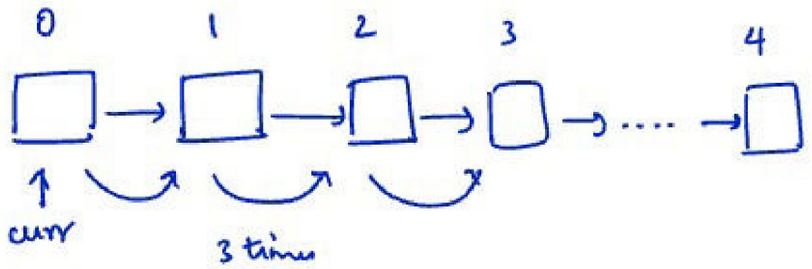
- RT is $O(n)$ ✓
- RT is $O(1)$ ✗
- RT is $\Theta(n)$ ✗
- RT is $\Theta(1)$ ✗
- RT is $\Omega(n)$ ✗
- RT is $\Omega(1)$ ✓

- Worst case RT is $O(n)$ ✓
- Worst case RT is $O(1)$ ✗
- Worst case RT is $\Theta(n)$ ✓
- Worst case RT is $\Theta(1)$ ✗
- Worst case RT is $\Omega(n)$ ✓
- Worst case RT is $\Omega(1)$ ✓

removeLast()

- RT is $\Theta(n)$ ✓

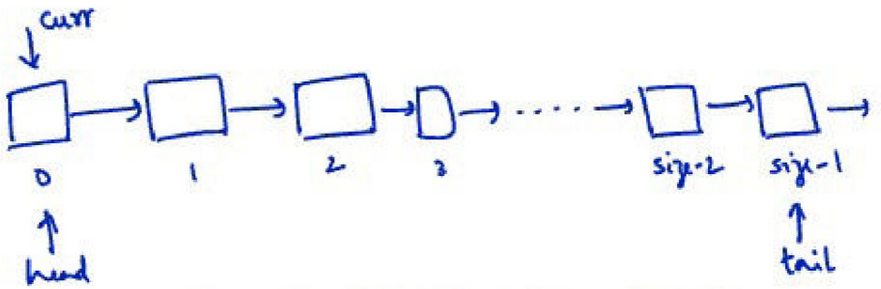
```
for (i=0; i<k; i++) {
    ...
}
```
runs k times



remove (index)

things that could fail:

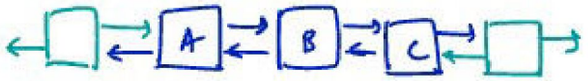
- invalid index - valid index range: [0, size-1]
- removing head (index 0) → head needs to be updated
- removing tail (index size-1) → tail needs to be updated
- single element list → both need to be null.



- go to node at index-1 & update its next.

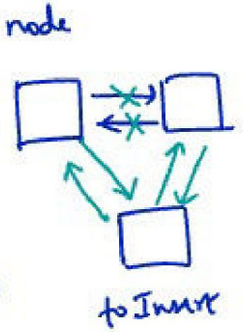
SLL - there was a lot of code for handling special cases

- Sentinel nodes (DLL)
  - These are dummy nodes before the start & after the end of the list

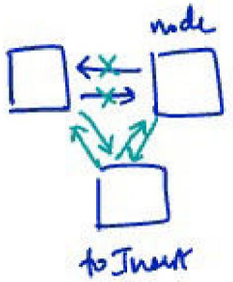


- insertAfter(node, toInsert)
  insertBefore(node, toInsert)
  deleteNode(node)

we can assume that node.next is not going to be null because we are using sentinel nodes

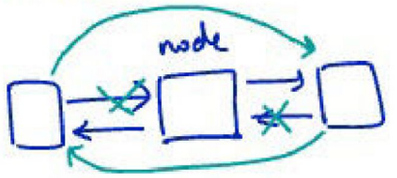


toInsert.next = node.next
node.next = toInsert
toInsert.prev = node
toInsert.next.prev = toInsert



toInsert.prev = node.prev
node.prev = toInsert
toInsert.next = node
toInsert.prev.next = toInsert

deleteNode(node)

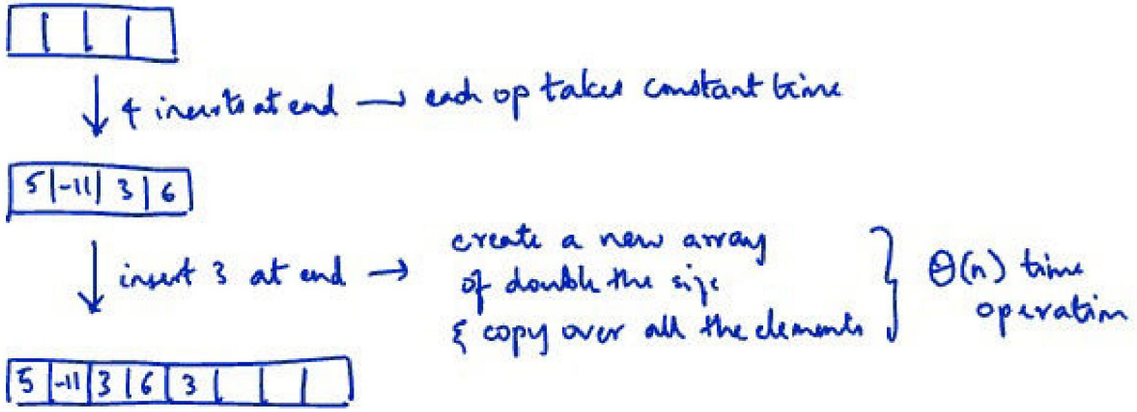


node.prev.next = node.next
node.next.prev = node.prev

Arrays: Fixed size & size has to be known beforehand.

Dynamic Arrays: solve this issue

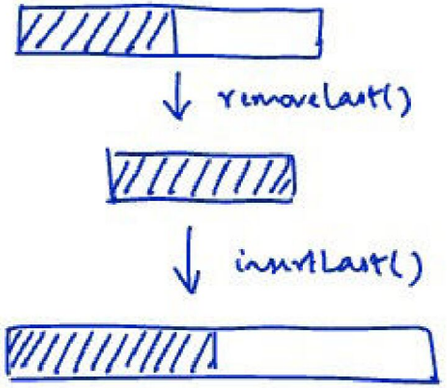
Idea: resize array whenever its full.



- Worst case running time for insertLast() is $\Theta(n)$ when array has to be resized. But it's too harsh as this scenario occurs rarely & predictably.

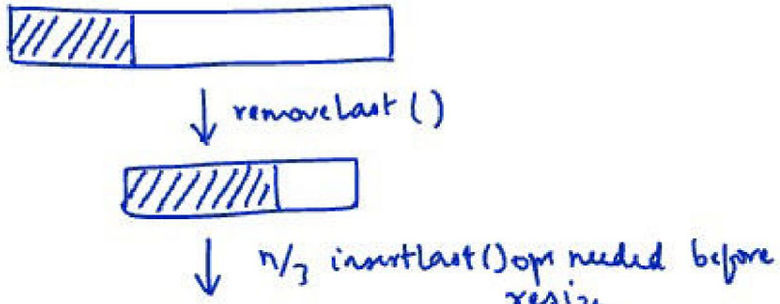
- Amortized running time:
  - Divide the work of a set operations & distribute to each operation
  - Intuition: If there are $\Theta(n)$ #ops that take constant time between two ops that each take $\Theta(n)$ time, then amortized running time of that op is $\Theta(1)$.

- Similarly, we could resize it down when it reaches a certain factor of the capacity
  - Scenario: We half the array if #elements become half the capacity.



There aren't $\Theta(n)$ #ops that take constant time between the two $\Theta(n)$ ops. So Amortized RT is not $\Theta(1)$ anymore

- Solution:
  - We could half the array when factor reaches (say) 1/3rd of capacity.



- Resize during add:

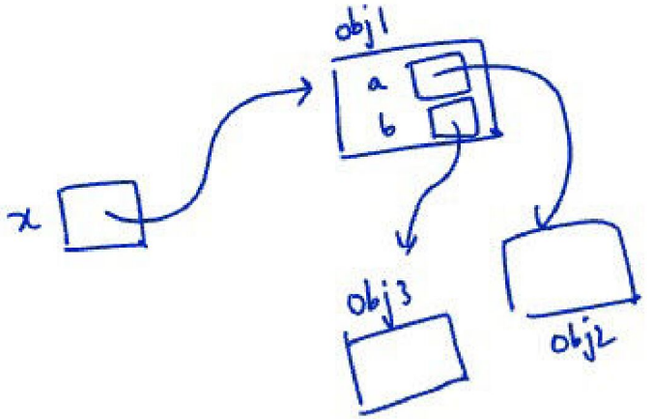
  Option 1: check before adding & resize if full

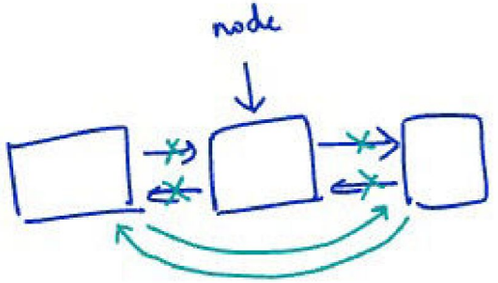
  Option 2: after adding an element, check & resize if full

  → resizes only when necessary

  → resizes beforehand & we might not insert a new element.

- Garbage collection






```
Node x = new Node(1);
x = new Node(2);
```

GP, AP

↓

$1+t+t^2+\cdots t^{n-1}$

$1+t+t^2+\cdots \infty$

$t+2t+3t^2+4t^3+\cdots \infty$

Amortized running time for 'n' insert operations

do comments & commit chunks & messages sincerely

AP:

$S = 1 + 2 + 3 + \cdots + (n-1) + n$

$S = n + (n-1) + (n-2) + \cdots + 2 + 1$

$2S = (n+1) + (n+1) + (n+1) + \cdots + (n+1) + (n+1)$

$2S = n(n+1)$

$\Rightarrow S = \frac{n(n+1)}{2}$

$T_1 \quad T_2 \quad T_3 \quad T_n$

$S = a + (a+d) + (a+2d) + \cdots + a+(n-1)d$

$S = a+(n-1)d + (a+(n-2)d) + (a+(n-3)d) + \cdots + a$

$2S = [2a+(n-1)d] + [2a+(n-1)d] + \cdots + [2a+(n-1)d]$

$= n[2a+(n-1)d]$

$\Rightarrow S_n = \frac{n}{2}[2a+(n-1)d]$

$T_n = a+(n-1)d$

G.P: Geometric Progression

- a - first term
- r - common ratio

$T_1 \quad T_2 \quad T_3 \quad T_n$

$a, ar, ar^2, \cdots, ar^{n-1}$

$S = a + ar + ar^2 + \cdots + ar^{n-1}$

$rS = ar + ar^2 + ar^3 + \cdots + ar^{n-1} + ar^n$

$S - Sr = a - ar^n$

$S = \frac{a(1-r^n)}{1-r}$

Assume $r < 1$

$S_n = \frac{a(1-r^n)}{(1-r)}$

$S_\infty = \lim_{n\to\infty} S_n = \lim_{n\to\infty} \frac{a(1-r^n)}{(1-r)} = \frac{a(1-0)}{(1-r)} = \frac{a}{1-r}$

$\boxed{S_\infty = \frac{a}{1-r}}$ if $r<1$

each step, fill half of remaining space

$\Rightarrow 1 + 1/2 + 1/4 + \cdots \infty = 2$

$r = 0.7$

$0.7^1 = 0.7$

$0.7^{10} = 0.028$

$0.7^{50} = 1.8 \times 10^{-10}$